

Hotspot Analysis

Andrea Cattarinich S5137057

November 10, 2025

1 Hotspot Analysis

The goal of this analysis is to identify the parts of the program that consume the most execution time. The measurements were performed using two methods:

- **Manually:** using `omp_get_wtime()`
- **Profiler:** using Intel VTune

Either two methods have been used with with $[N = 40000]$ on a Room-210 Workstation and produced similar results.

Table 1: Manually

Function	Time (s)	Percentage (%)
fillInput	0.0053	0.03
setOutputZero1	0.0005	0.00
setOutputZero2	0.0005	0.00
DFT	8.0331	49.97
IDFT	8.0345	49.99
checkResults	0.0000	0.00
Total	16.0741	100.00

Table 2: Intel VTune Profiler

Function Stack	CPU Time: Self	Percentage (%)
Total	16.06	100.00
._start	16.06	0.00
._libc.start_main_impl	16.06	0.00
- main	16.06	0.00
— DFT	8.032	14.21
— __svml.sincos2	5.74977	1.72
— __svml.sincos2_l9	5.47389	34.08
— DFT	8.028	14.02
— __svml.sincos2	5.77596	1.54
— __svml.sincos2_l9	5.52797	34.42

 **Top Hotspots**
This section lists the most active functions in your application. Optimizing these hotspot functions typically results in improving overall application performance.

Function	Module	CPU Time 	% of CPU Time 
__svml_sincos2_l9	omp_homework_hotspots_v2	11.002s	68.5%
DFT	omp_homework_hotspots_v2	2.282s	14.2%
DFT	omp_homework_hotspots_v2	2.252s	14.0%
__svml_sincos2	omp_homework_hotspots_v2	0.524s	3.3%

*N/A is applied to non-summable metrics.

1.1 Conclusion

The main hotspot is the DFT() function and its inverse IDFT(), which together account for over 99% of the total execution time. All other functions have negligible impact. This indicates that **parallelization and vectorization efforts should focus on the double loop inside DFT()**.

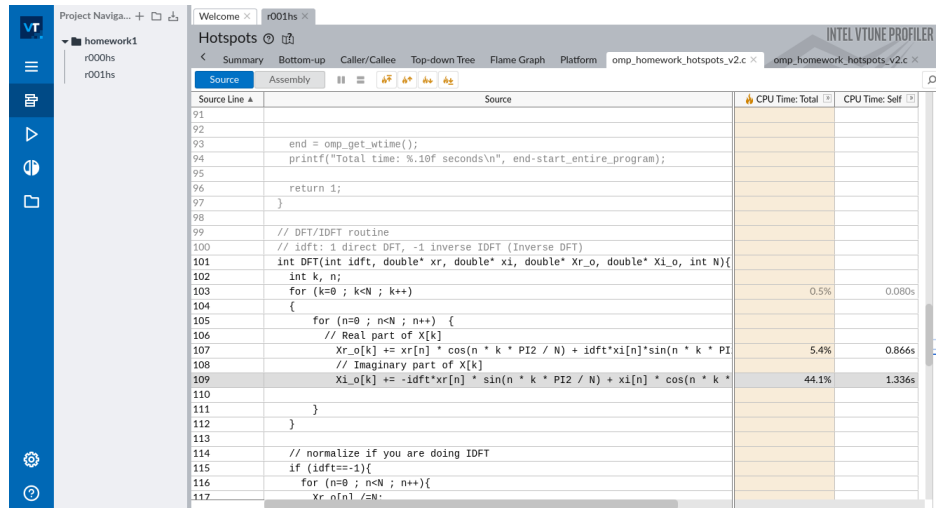


Figure 1: The main hotspot is the loop contained in lines 103-112.